

Thailand AI Live + Labs 26 - QueryData for AlloyDB using Gemini Data Analytics

來源：<https://codelabs.developers.google.com/th-ai-live-labs-26-alloydb-querydata?hl=zh-tw>

步驟數：14

在本程式碼研究室中，您將瞭解如何在範例代理程式應用程式中，搭配 AI 代理程式使用 AlloyDB 適用的 Google Cloud MCP 服務。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 事前準備](#)
4. [4. 部署 AlloyDB](#)
5. [5. 準備資料庫](#)
6. [6. 範例資料](#)
7. [7. 使用資料代理](#)
8. [8. 測試不含 QueryData 的 AlloyDB NL2SQL](#)
9. [9. 建立 QueryData ContextSet](#)
10. [10. 測試 QueryData 脈絡資料集](#)
11. [11. 使用 AI 代理查詢資料](#)
12. [12. 清理環境](#)
13. [13. 恭喜](#)
14. [14. 問卷調查](#)

1. 簡介

本實驗室已修改為使用曼谷區域，以配合 2026 年泰國 AI Live + Labs 活動。原始實驗室請見[這裡](#)

本程式碼實驗室將說明如何開始使用 AlloyDB 的 QueryData，並在代理程式應用程式中，透過自然語言輸入生成準確且可預測的 SQL 陳述式。

必要條件

- Google Cloud 控制台的基本知識
- 指令列介面和 Cloud Shell 的基本技能

課程內容

- 如何建立 AlloyDB 叢集並匯入範例資料
- 如何[啟用](#) AlloyDB 資料存取 API
- 如何為 AlloyDB 啟用 QueryData
- 如何生成範本
- 如何使用多面向搜尋
- 如何搭配 AI 代理使用 QueryData

軟硬體需求

- Google Cloud 帳戶和 Google Cloud 專案
 - 支援 Google Cloud 控制台和 Cloud Shell 的網路瀏覽器，例如 [Chrome](#)
-

步驟 2 · 約 5 分鐘

2. 設定和需求

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

專案設定

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上角工具列的 Cloud Shell 圖示：



你也可以依序按下 G 和 S 鍵。如果您位於 Google Cloud 控制台中，或使用這個[連結](#)，這個序列會啟動 Cloud Shell。

佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell terminal. The browser tab is titled 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal has a dark background and shows the text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by a prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 3 · 約 5 分鐘

3. 事前準備

啟用 API

請注意，啟用部分資源會產生費用。在正常情況下，如果實驗室完成後 AlloyDB 執行個體和運算 VM 都遭到毀損，所有資源的費用不會超過 \$5 美元。建議您查看帳單，確認這項措施是否可接受。

如要使用 [AlloyDB](#)、[Compute Engine](#)、[網路服務](#)和 [Vertex AI](#)，您需要在 Google Cloud 雲端專案中啟用各自的 API。

在 Cloud Shell 終端機中，確認專案 ID 已設定完畢：

```
gcloud config get-value project
```

輸出內容應會顯示專案 ID：

```
student@cloudshell:~ (test-project-001-402417)$ gcloud config get-value project
Your active configuration is: [cloudshell-23188]
test-project-001-402417
student@cloudshell:~ (test-project-001-402417)$
```

設定環境變數 PROJECT_ID：

```
PROJECT_ID=$(gcloud config get-value project)
```

啟用所有必要服務：

```
gcloud services enable alloydb.googleapis.com \
    compute.googleapis.com \
    cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    geminidataanalytics.googleapis.com \
    cloudaicompanion.googleapis.com \
    aiplatform.googleapis.com
```

預期的輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ gcloud config set project test-
project-001-402417
Updated property [core/project].
student@cloudshell:~ (test-project-001-402417)$ PROJECT_ID=$(gcloud config get-value
project)
Your active configuration is: [cloudshell-14650]
student@cloudshell:~ (test-project-001-402417)$
student@cloudshell:~ (test-project-001-402417)$ gcloud services enable
alloydb.googleapis.com \
    compute.googleapis.com \
    cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    geminidataanalytics.googleapis.com \
    cloudaicompanion.googleapis.com \
    aiplatform.googleapis.com
Operation "operations/acat.p2-4470404856-1f44ebd8-894e-4356-bea7-b84165a57442"
finished successfully.
```

步驟 4 · 約 10 分鐘

4. 部署 AlloyDB

建立 AlloyDB 叢集和主要執行個體。您可以透過準備好的指令碼部署，該指令碼會部署所有必要資源，也可以按照[說明文件](#)逐步部署。

使用自動化指令碼部署 AlloyDB

這個方法會使用自動化指令碼部署 AlloyDB 叢集，並提供必要資訊，以便開始使用已部署的資源。

這個指令碼適用於預設網路，可為 AlloyDB 私人 IP 建立必要元件。如果您有自訂網路設定，並打算使用不同的叢集名稱和分配，請參閱 AlloyDB 文件中的指南。

在 Cloud Shell 終端機執行指令，從存放區複製部署指令碼。

```
REPO_NAME="codelabs"  
REPO_URL="https://github.com/GoogleCloudPlatform/$REPO_NAME"  
SOURCE_DIR="alloydb-querydata"  
  
git clone --no-checkout --filter=blob:none --depth=1 $REPO_URL  
  
cd $REPO_NAME  
git sparse-checkout set $SOURCE_DIR  
git checkout  
cd $SOURCE_DIR
```

執行部署指令碼。

```
./deploy_alloydb.sh --region=asia-southeast3 --public-ip
```

指令碼需要一些時間才能執行完畢 (通常約 5 到 7 分鐘)，並部署 AlloyDB 叢集和具有公開與私人 IP 的主要執行個體。公開 IP 僅適用於授權網路，或透過 AlloyDB Auth Proxy 使用。如要進一步瞭解公用 IP，請參閱[說明文件](#)。指令碼輸出內容應提供已部署 AlloyDB 叢集的相關資訊。請注意，密碼會有所不同，請記下密碼以供日後使用。

```
...
<redacted>
...
Creating primary instance: alloydb-aip-01-pr (8 vCPUs for TRIAL cluster)
Operation ID: operation-1765988049916-646282264938a-bddce198-9f248715
Creating instance...done.

-----

Deployment Process Completed
Cluster: alloydb-aip-01 (TRIAL)
Instance: alloydb-aip-01-pr
Region: asia-southeast3
Initial Password: JBBoDTgixzYwYpkF (if new cluster)

-----
```

您也可以在網頁[控制台](#)中查看新叢集和主要執行個體

 Clusters [+ Create cluster](#) [Migrate data](#) [Install AlloyDB Omni](#) [Purchase AlloyDB Omni](#)

Improve instance health by investigating issues and acting on recommendations. [^ Collapse details](#) [View fleet health](#)

Data protection

1 No backup policy enabled →

[Show affected resources](#)

Clusters are groups of instances of virtual machines that power your workload, and can include a primary instance and multiple read pool instances. All cluster resources share a storage layer, which scales to suit your needs. [Learn more](#)

Filter Enter property name or value

Resource name ↑	Resource type	Location	Read pool nodes	Maintenance	Issues	Private IP address
▼ alloydb-aip-01	Primary cluster	us-central1			No backup policy enabled →	▼
✓ alloydb-aip-01-pr	Primary instance	us-central1-c				172.19.96.2:5432 📄

步驟 5 · 約 10 分鐘

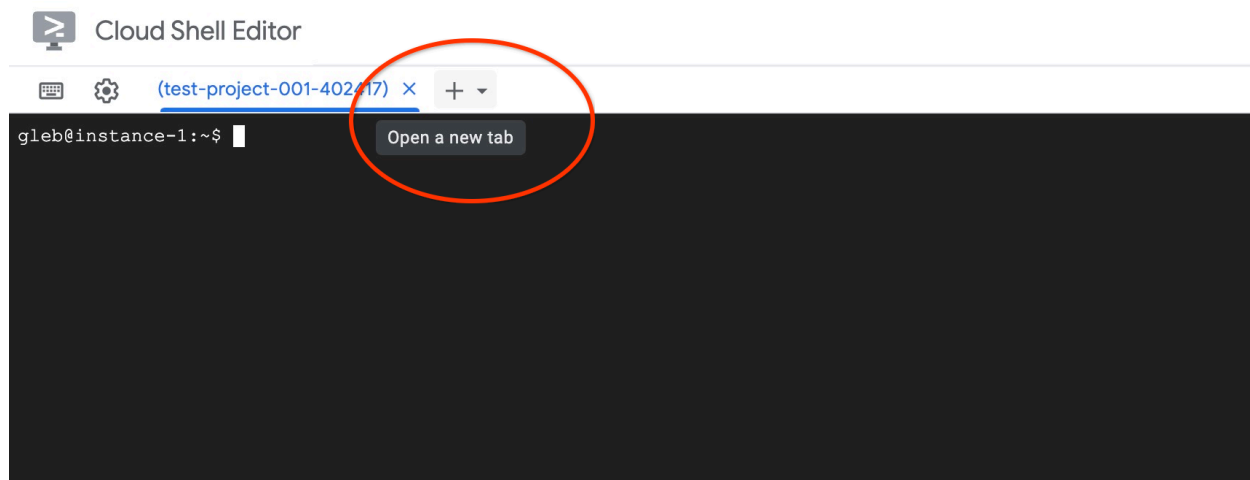
5. 準備資料庫

如要使用 AI 函式和運算子，請啟用 Vertex AI 整合功能、資料存取 API，並為範例資料集建立資料庫。

授予 AlloyDB 必要權限

將 Vertex AI 權限新增至 AlloyDB 服務代理。

使用頂端的「+」符號開啟另一個 Cloud Shell 分頁。



在新開啟的 Cloud Shell 分頁中執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

預期的控制台輸出內容：

```

student@cloudshell:~ (test-project-001-402417)$ PROJECT_ID=$(gcloud config get-value
project)
Your active configuration is: [cloudshell-11039]
student@cloudshell:~ (test-project-001-402417)$ gcloud projects add-iam-policy-
binding $PROJECT_ID \
    --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \
    --role="roles/aiplatform.user"
Updated IAM policy for project [test-project-001-402417].
bindings:
- members:
  - serviceAccount:service-4470404856@gcp-sa-alloydb.iam.gserviceaccount.com
    role: roles/aiplatform.user
- members:
  ...
etag: BwYIEbe_Z3U=
version: 1

```

啟用 Data Access API

您必須在 AlloyDB 叢集上啟用 Data Access API，才能使用 `execute_sql` 等 MCP 工具。

在同一個終端機分頁中執行。

```

PROJECT_ID=$(gcloud config get-value project)
REGION=asia-southeast3
ADBCLUSTER=alloydb-aip-01
curl -X PATCH \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  -H "Content-Type: application/json" \
  https://alloydb.googleapis.com/v1alpha/projects/$PROJECT_ID/locations/$REGION/clusters/$ADBC
LUSTER/instances/$ADBCLUSTER-pr?updateMask=dataApiAccess \
  -d '{
    "dataApiAccess": "ENABLED",
  }'

```

啟用 IAM 驗證

我們將為代理工具使用 IAM 驗證功能，因此必須在執行個體上啟用 IAM 驗證，並將自己新增為資料庫使用者。請先完成上一個步驟，啟用資料存取權 API，再啟用執行個體層級的 IAM 驗證。執行個體狀態應為綠色。

Clusters are groups of instances of virtual machines that power your workload, and can include a primary instance and multiple read pool instances. All cluster resources share a storage layer, which scales to suit your needs. [Learn more](#)

Filter Enter property name or value

Resource name ↑	Resource type	Location	Read pool nodes	Maintenance
▼ alloydb-aip-01	Primary cluster	us-central1		
▼ alloydb-aip-01-pr	Primary instance	us-central1-a		

首先，我們要在執行個體層級啟用 IAM。在同一個終端機分頁中執行。

```
PROJECT_ID=$(gcloud config get-value project)
REGION=asia-southeast3
ADBCLUSTER=alloydb-aip-01
gcloud beta alloydb instances update $ADBCLUSTER-pr \
  --database-flags password.enforce_complexity=on,alloydb.iam_authentication=on \
  --region=$REGION \
  --cluster=$ADBCLUSTER \
  --project=$PROJECT_ID \
  --update-mode=FORCE_APPLY
```

將自己新增為 AlloyDB 使用者：

```
REGION=asia-southeast3
ADBCLUSTER=alloydb-aip-01
gcloud alloydb users create $(gcloud config get-value account) \
  --cluster=$ADBCLUSTER \
  --superuser=true \
  --region=$REGION \
  --type=IAM_BASED
```

在分頁中執行「exit」指令，關閉分頁：

```
exit
```

連線至 AlloyDB Studio

在接下來的章節中，所有需要連線至資料庫的 SQL 指令，都可以在 AlloyDB Studio 中執行。T

前往 [PostgreSQL 適用的 AlloyDB 的「叢集」頁面](#)。

按一下主要執行個體，開啟 AlloyDB 叢集的網頁控制台介面。

Clusters

[CREATE CLUSTER](#) [MIGRATE DATABASE](#) [INSTALL ALLOYDB OMNI](#) [PURCHASE ALLOYDB OMNI](#)

Clusters are groups of instances of virtual machines that power your workload, and can include a primary instance and multiple read pool instances. All cluster resources share a storage layer, which scales to suit your needs. [Learn more](#)

Filter Enter property name or value

Resource name ↑	Resource type	Location	Read pool nodes	Issues	Private IP address	High availability
alloydb-aip-01	Primary cluster	us-central1				Highly available (multi-zone)
alloydb-aip-01-pr	Primary instance	us-central1-a			10.38.74.2:5432	Highly available (multi-zone)

然後按一下左側的 AlloyDB Studio：

AlloyDB

PRIMARY CLUSTER

Overview

AlloyDB Studio

System insights

Query insights

Data protection

Users

Status Ready

Cluster ID alloydb-aip-01

Version PostgreSQL 15 compatible

Cluster type Primary cluster

High availability Highly available (multi-zone)

Location us-central1 (Iowa) Low CO2

Total storage used 104.2 MB

Network default

選擇 postgres 資料庫和 IAM 驗證。然後按一下「驗證」按鈕。

All clusters > Primary cluster alloydb-aip-01: AlloyDB Studio > Primary instance alloydb-aip-01-pr

Explorer

Untitled query

Welcome to AlloyDB Studio

You can run queries, create views, modify schemas, and more, all from one surface. Read the quickstart guide for the basics of how to compose a new query, or check out the documentation for more details on setting the most out of your AlloyDB cluster.

No objects to display

Log in to your database

Choose a database and user account to securely access your data. [Learn more](#)

Instance alloydb-aip-01-pr

Database * postgres

Authentication method

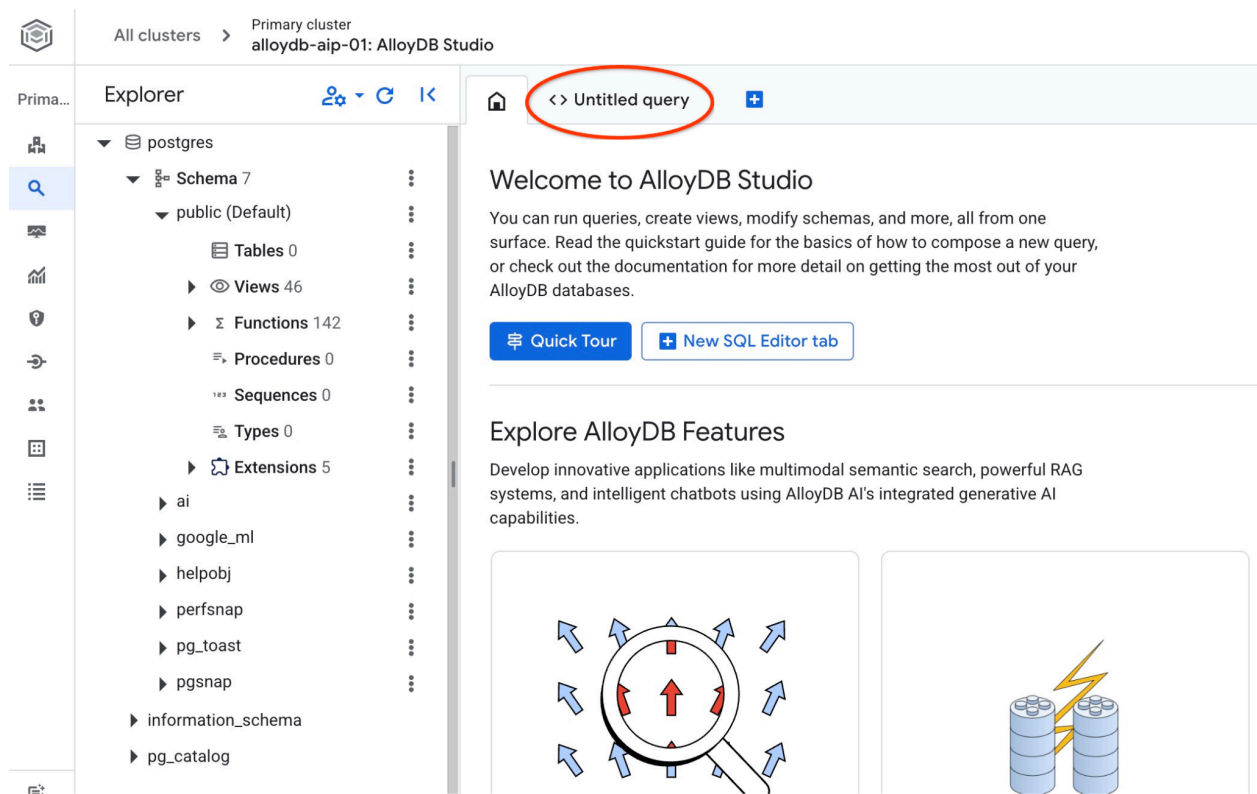
Manage users and passwords from the [users page](#)

☒ IAM database authentication

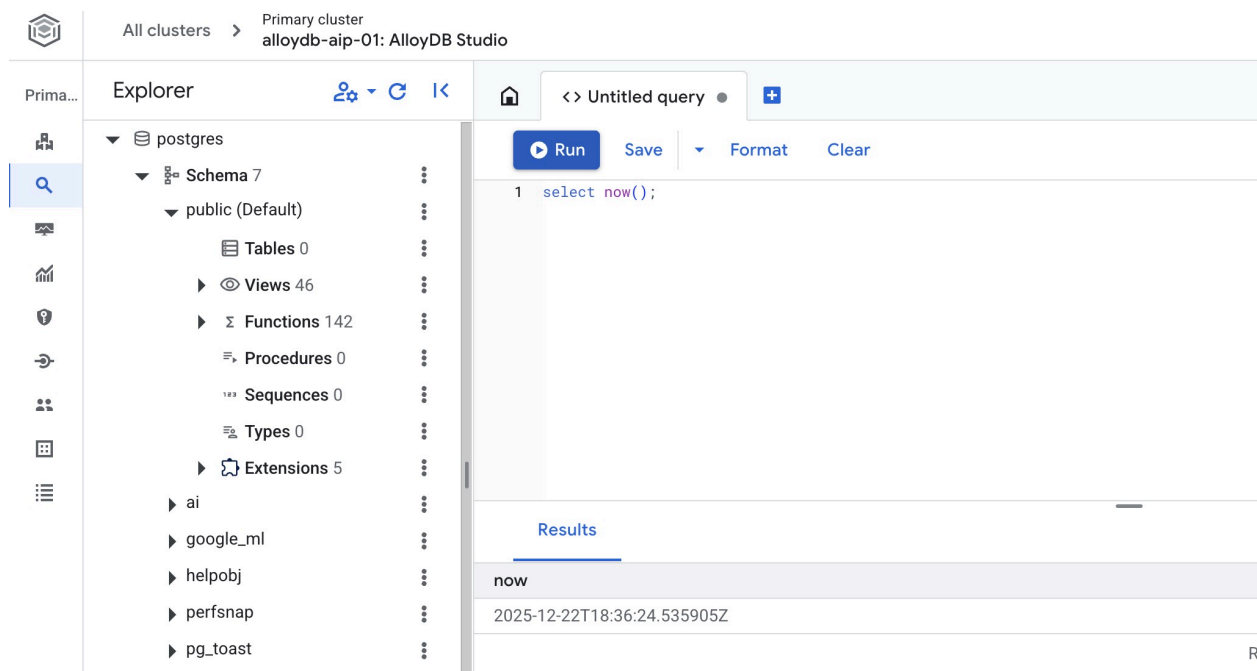
☐ Built-in database authentication

Authenticate Cancel

AlloyDB Studio 介面隨即開啟。如要在資料庫中執行指令，請按一下右側的「Untitled Query」(未命名查詢) 分頁。



開啟介面，供您執行 SQL 指令



建立資料庫

建立資料庫快速入門。

在 AlloyDB Studio 編輯器中執行下列指令。

建立資料庫：

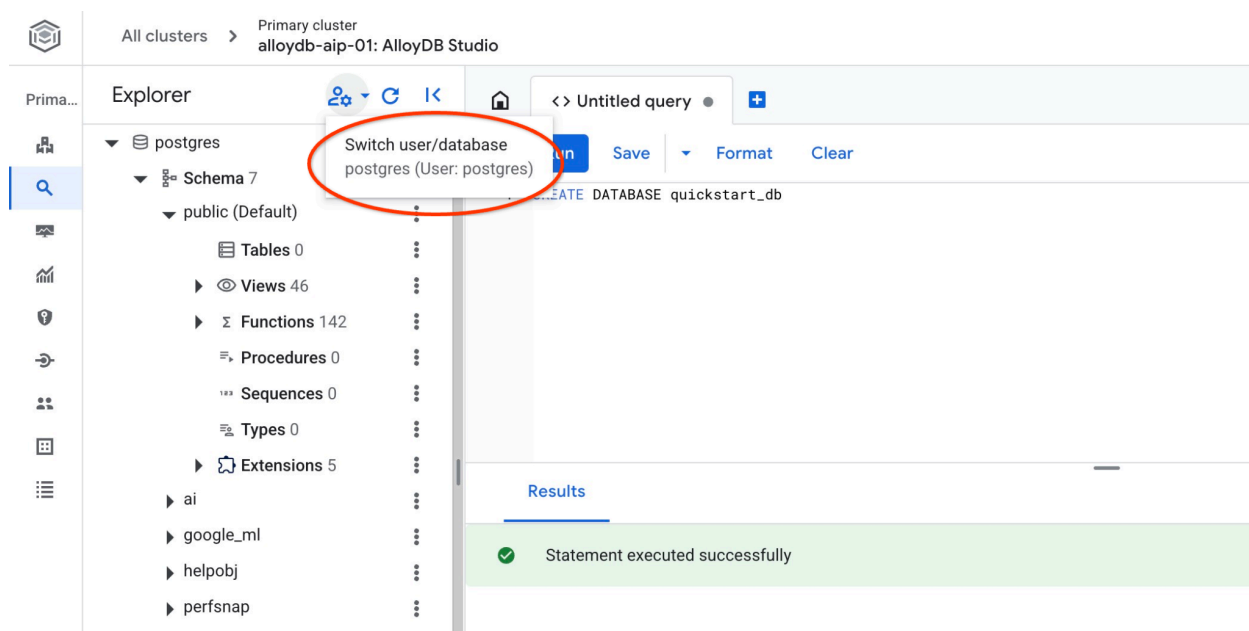
```
CREATE DATABASE quickstart_db
```

預期輸出內容：

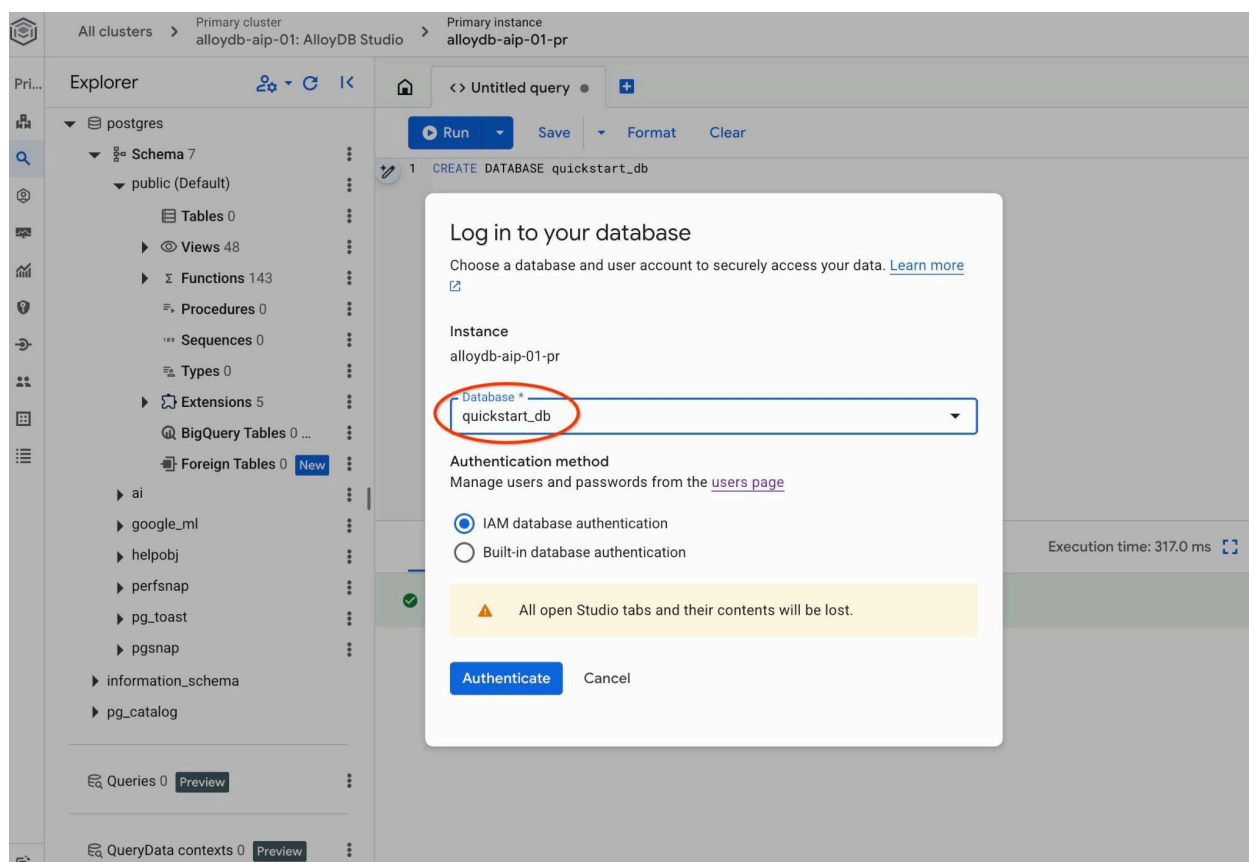
Statement executed successfully

連線至 quickstart_db

連線至資料庫，確認資料庫是否已建立。使用切換使用者/資料庫的按鈕，重新連線至工作室。



從下拉式清單中選取新的 quickstart_db 資料庫，並使用相同的 IAM 驗證。



系統會開啟新連線，讓您處理 `quickstart_db` 資料庫中的物件。您可以在該處檢查匯入的結構定義和資料，並使用 QueryData 環境集。

步驟 6 · 約 5 分鐘

6. 範例資料

現在您需要在資料庫中建立物件並載入資料。您將使用虛構的 Cymbal Shipping 公司資料集。其中包含貨物、卡車、要求和卡車行程的虛構資料，以及虛構的司機。

建立 Storage bucket

您將使用 Google SDK (gcloud)，將複製存放區的資料匯入 AlloyDB 資料庫。您需要為此建立 Cloud Storage 值區，並授予 AlloyDB 服務帳戶存取權。或者，您也可以隨時嘗試使用網頁控制台執行這項操作，詳情請參閱[說明文件](#)。

在 Google Cloud Shell 終端機執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)
REGION=asia-southeast3
gcloud storage buckets create gs://$PROJECT_ID-import --project=$PROJECT_ID --location=$REGION
gcloud storage buckets add-iam-policy-binding gs://$PROJECT_ID-import --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" --role=roles/storage.objectViewer
```

載入資料

下一步是載入資料。壓縮的 SQL 傾印檔位於複製的存放區資料夾中。下列指令假設您在建立 AlloyDB 叢集時，已使用主目錄做為起點，複製了上一步驟中的存放區。

將壓縮的 SQL 傾印檔複製到新的儲存空間 bucket：

```
REPO_NAME="codelabs"
SOURCE_DIR="alloydb-querydata"
cd ~/$REPO_NAME/$SOURCE_DIR
gcloud storage cp ~/$REPO_NAME/$SOURCE_DIR/postgres_dump.sql.gz gs://$PROJECT_ID-import
```

然後將資料載入 `quickstart_db` 資料庫：

```
PROJECT_ID=$(gcloud config get-value project)
CLUSTER_NAME=alloydb-aip-01
REGION=asia-southeast3
gcloud alloydb clusters import $CLUSTER_NAME --region=asia-southeast3 --database=quickstart_db --gcs-uri=gs://$PROJECT_ID-import/postgres_dump.sql.gz --project=$PROJECT_ID --sql
```

這項指令會將範例資料集載入 `quickstart_db` 資料庫。您可以使用 AlloyDB Studio 驗證資料表和記錄。

7. 使用資料代理

首先，請使用 Google ADK for Python 建立範例 AI 代理程式，並使用 MCP Toolbox for Databases 連線至 AlloyDB 執行個體。

安裝 MCP Toolbox for Databases

資料庫適用的 MCP 工具箱是開放原始碼專案，可為多個資料庫引擎提供 MCP 支援，包括 AlloyDB for PostgreSQL。如要瞭解 MCP Toolbox，請參閱[說明文件](#)。

請為您的平台下載最新版軟體。如要查看最新版本，請前往版本資訊頁面。以下範例說明如何將 MCP Toolbox 第 31 版下載至 Cloud Shell。

```
REPO_NAME="codelabs"
SOURCE_DIR="alloydb-querydata"
cd ~/$REPO_NAME/$SOURCE_DIR
export VERSION=0.31.0
curl -L -o toolbox https://storage.googleapis.com/genai-toolbox/v$VERSION/linux/amd64/toolbox
chmod +x toolbox
```

您需要準備 Toolbox 的設定檔。我們在目前目錄中有範例 `tools.yaml.example` 檔案，並將準備 `tools.yaml` 檔案，方法是將兩個預留位置替換為專案 ID 和區域。

```
PROJECT_ID=$(gcloud config get-value project)
REGION=asia-southeast3
sed -e "s/##PROJECT_ID##/$PROJECT_ID/g" \
    -e "s/##REGION##/$REGION/g" \
    tools.yaml.example > tools.yaml
```

啟動 MCP Toolbox for Databases

現在可以使用準備好的設定檔啟動 MCP 工具箱。

在 Google Cloud Shell 介面頂端按下「+」按鈕，開啟 Google Cloud Shell 的新分頁。

在新分頁中，切換至含有工具箱二進位檔案和設定檔 `tools.yaml` 的目錄，然後啟動 MCP 伺服器。

```
REPO_NAME="codelabs"
SOURCE_DIR="alloydb-querydata"
cd ~/$REPO_NAME/$SOURCE_DIR
./toolbox --config tools.yaml
```

輸出內容應會顯示「Server ready to serve!」，如下所示。

```
2026-03-30T10:28:03.614374-04:00 INFO "Initialized 1 sources: cymbal-logistics-sql-source"
2026-03-30T10:28:03.614517-04:00 INFO "Initialized 0 authServices: "
2026-03-30T10:28:03.614531-04:00 INFO "Initialized 0 embeddingModels: "
2026-03-30T10:28:03.614657-04:00 INFO "Initialized 2 tools: execute_sql_tool, list_cymbal_logistics_schemas_tool"
2026-03-30T10:28:03.614711-04:00 INFO "Initialized 1 toolsets: default"
2026-03-30T10:28:03.614723-04:00 INFO "Initialized 0 prompts: "
2026-03-30T10:28:03.614779-04:00 INFO "Initialized 1 promptsets: default"
2026-03-30T10:28:03.616214-04:00 INFO "Server ready to serve!"
```

檢查代理程式原始碼

在複製的存放區資料夾的第一個分頁中，使用 Google Cloud Shell 編輯器檢查代理程式程式碼。

```
REPO_NAME="codelabs"
SOURCE_DIR="alloydb-querydata"
edit ~/${REPO_NAME}/${SOURCE_DIR}/data_agent/agent.py
```

您可以在代理程式中看到 AlloyDB 適用的 Google Cloud MCP 伺服器專區。我們提供端點做為 MCP_SERVER_URL、驗證、專案 ID，並將其新增至 MCP 工具集。

```
MCP_SERVER_URL = "http://127.0.0.1:5000"
creds, project_id = default(scopes=["https://www.googleapis.com/auth/cloud-platform"])
if not creds.valid:
    creds.refresh(GoogleAuthRequest())

print(f"Authenticated as project: {project_id}")

mcp_toolset = ToolboxToolset(
    server_url=MCP_SERVER_URL,
)
```

在代理程式碼中，MCP 工具組會做為代理程式的 `tools` 參數。此外，叢集和執行個體名稱、區域和資料庫也會做為代理程式提示的變數。

```

MODEL_ID = "gemini-3-flash-preview"
cluster_name="alloydb-aip-01"
instance_name="alloydb-aip-01-pr"
location="asia-southeast3"
database_name="quickstart_db"

# Agent configuration

root_agent = Agent(
    model=MODEL_ID,
    name='root_agent',
    description='A helpful assistant for analyst requests.',
    instruction=f"""
    Answer user questions to the best of your knowledge using provided tools.
    Do not try to generate non-existent data but use the grounded data from the
    database.

    When you answer questions about Cymbal Logistic activity
    use the toolset to run query in the AlloyDB cluster {cluster_name} instance
    {instance_name} in the location {location}
    in the project {project_id} in the database {database_name}
    """,
    tools=[mcp_toolset],
)

```

檢查程式碼後，按一下編輯器視窗右上方的「Open terminal」按鈕，切換回終端機。

啟動代理程式

現在，您可以使用 Google ADK 網頁介面，以互動模式啟動代理程式。ADK 網頁介面提供便利的方式，可測試及排解代理的工作流程問題。

首先，請使用 uv 套件管理工具，安裝 Python 的所有必要套件。

```

REPO_NAME="codelabs"
SOURCE_DIR="alloydb-querydata"
cd ~/$REPO_NAME/$SOURCE_DIR
uv sync

```

安裝所有套件後，您需要在代理程式目錄中新增 `.env` 檔案，引導代理程式使用 Vertex AI 與 AI 模型進行所有通訊。

```

echo "GOOGLE_GENAI_USE_VERTEXAI=true" > data_agent/.env
echo "GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project -q)" >> data_agent/.env
echo "GOOGLE_CLOUD_LOCATION=global" >> data_agent/.env

```

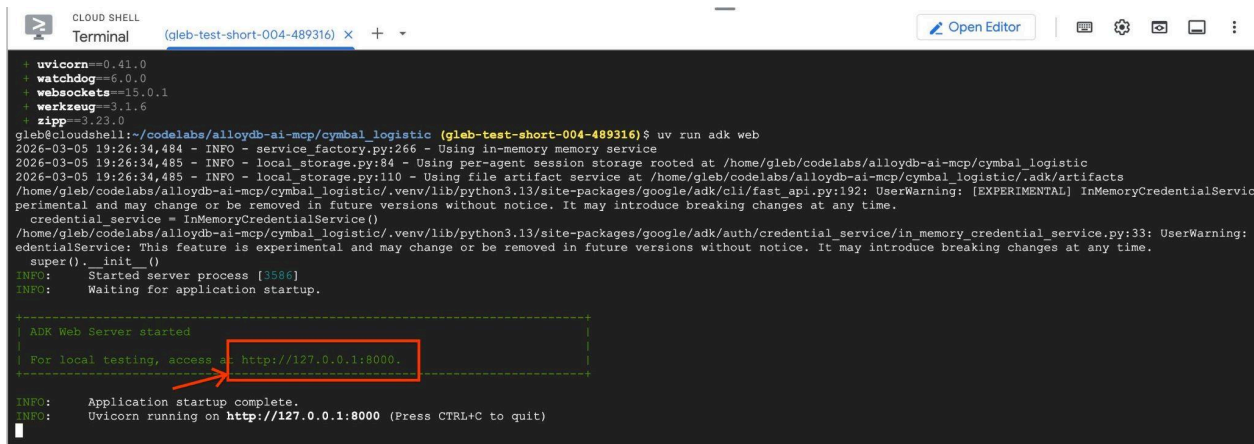
接著即可啟動代理

```

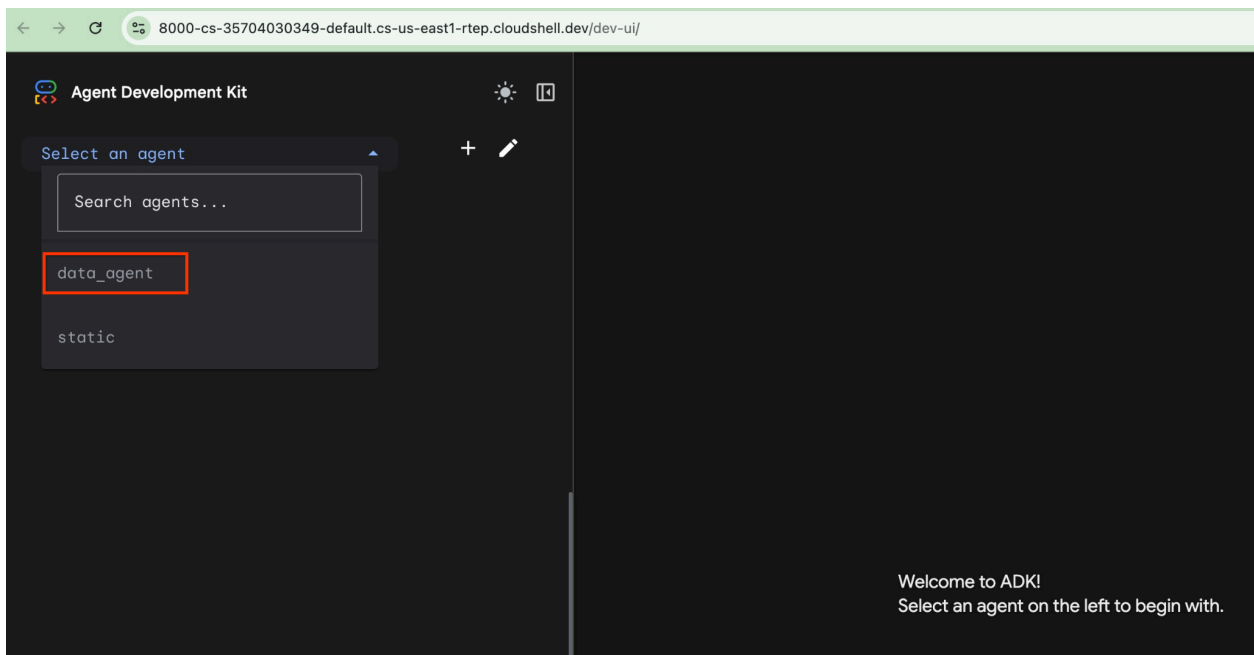
uv run adk web --allow_origins 'regex:https://.*\.cloudshell\.dev'

```

您應該會看到類似下列的輸出內容，其中包含 <http://127.0.0.1:8000> 等端點。



您可以點選 Cloud Shell 中的網址，系統會在另一個瀏覽器分頁中開啟預覽視窗，您可以在左側的下拉式清單中選擇 `data_agent` 。

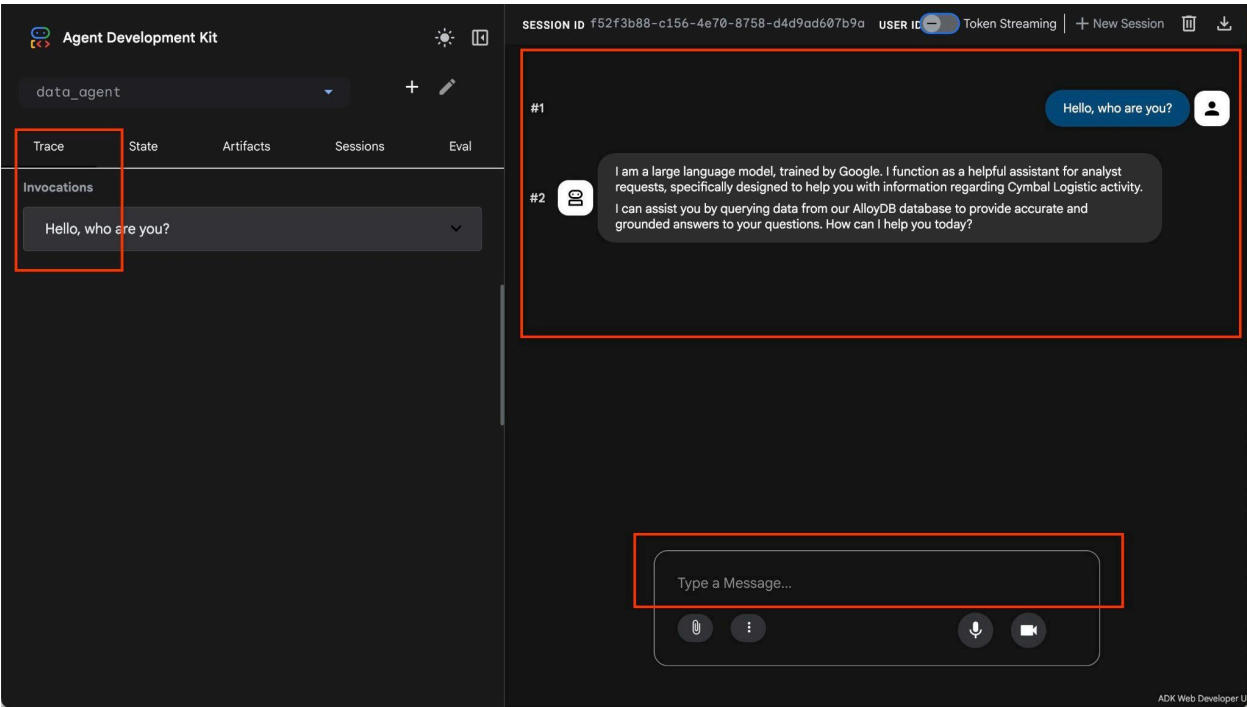


在 ADK 網頁介面中，您可以在右下方發布問題，並在右側查看完整執行流程，包括每個步驟的追蹤記錄。

步驟 8 · 約 10 分鐘

8. 測試不含 QueryData 的 AlloyDB NL2SQL

您可以透過自然語言以任意形式提問，代理程式會使用 MCP toolbox for databases 做為工具來回答問題。問題會顯示在右下角，而答案和所有工具呼叫則會顯示在頂端。

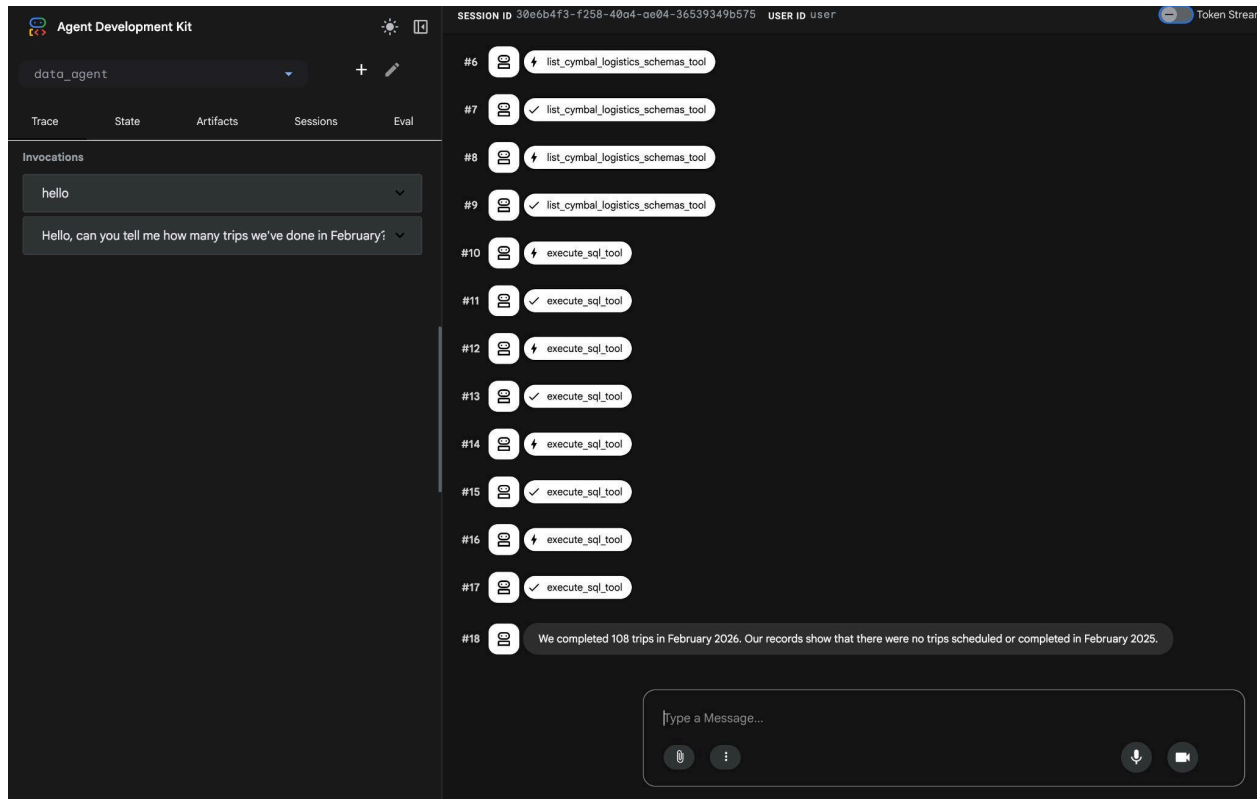


您正在處理貨運公司的營運資料，其中包含貨運要求、卡車、司機和司機行程的相關資訊。第一個問題是 2026 年 2 月的行程次數。

在右下方的輸入欄位中輸入下列內容，然後按下 Enter 鍵。

Hello, can you tell me how many trips we've done in February?

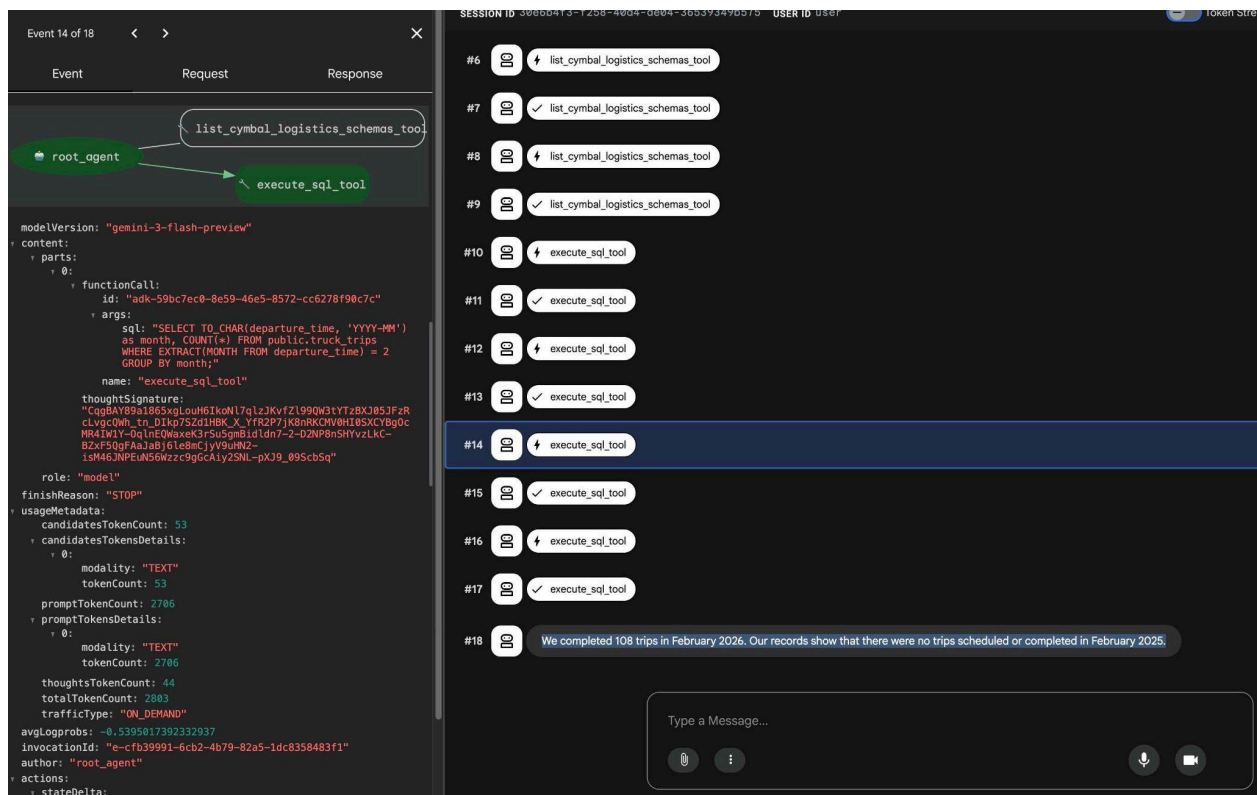
代理程式會執行多個工具呼叫，使用 `list_cymbal_logistics_schemas_tool` 和 `execute_sql_tool` 執行多個 SQL 陳述式，找出結構定義中的正確資料表，取得正確資料。



建立適當的查詢並在資料庫中執行後，最終會產生正確結果。

我們在 2026 年 2 月完成了 108 趟行程。記錄顯示，2025 年 2 月沒有任何預定或完成的行程。

點選工具執行作業，即可查看每個工具呼叫執行的動作。舉例來說，以下是執行查詢以取得結果的範例。



使用 ADK 網頁介面嘗試其他簡單要求，瞭解系統如何執行不同查詢來達成結果。

在終端機中按下 `ctrl+c` 鍵，停止代理程式。您可以關閉顯示 ADK 網頁介面的瀏覽器分頁。

你也可以在第二個分頁中按下相同的 `ctrl+c` 鍵快速鍵，停止 MCP 工具箱，然後關閉第二個分頁。

在下一個步驟中，我們會建構 QueryData 內容，以提升 NL2SQL 回應和效能。

9. 建立 QueryData ContextSet

您在上一個步驟中可以看到，AI 模型多次呼叫資料庫的資訊結構定義，以判斷應使用哪些資料表和資料欄來建構 SQL 查詢。為提升效能和準確率，並讓結果更具預測性，我們會加入 QueryData 內容，定義應執行哪些查詢來回應特定要求。

建立目標對象專屬範本

QueryData ContextSet 是 JSON 檔案，內含查詢範本和層面，可根據查詢模式和資料結構，為 AI 模型提供必要的資料和指示，確保模型使用正確的 SQL 查詢或 SQL 查詢部分，達成要求目標。

請先建立目標範本；使用 Cloud Shell 編輯器建立檔案。在 Cloud Shell 終端機中執行。

```
edit ~/$REPO_NAME/$SOURCE_DIR/data_agent/querydata_cymbal_contextset.json
```

然後插入上一章使用的自然語言查詢範本：「我們在 2 月完成了幾趟行程？」

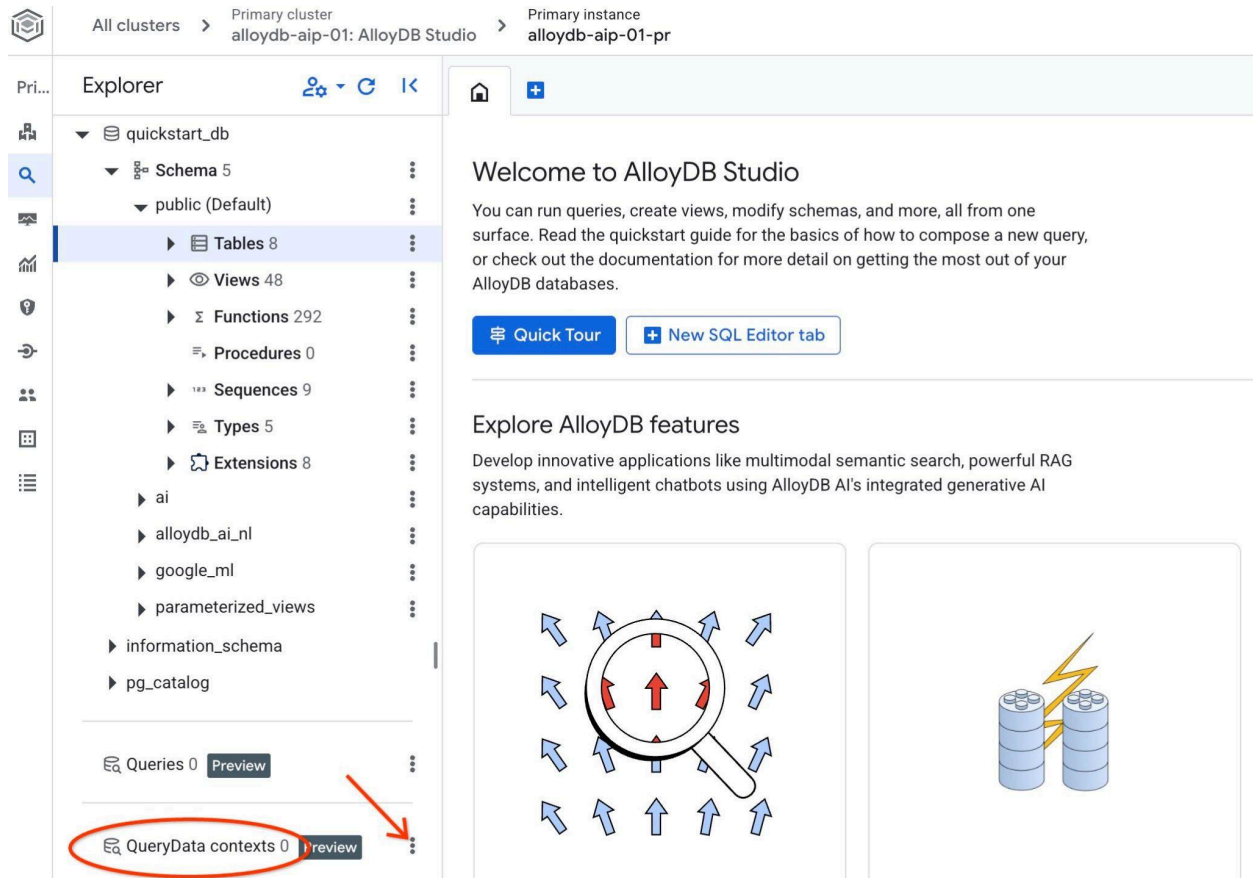
```
{
  "templates": [
    {
      "nl_query": "How many trips we've done in February?",
      "sql": "SELECT COUNT(*) FROM truck_trips WHERE departure_time >=TO_DATE('February 2026', 'Month YYYY') AND departure_time < TO_DATE('February 2026', 'Month YYYY') + INTERVAL '1 month'",
      "intent": "Count trips done in a given month like February 2026",
      "manifest": "How many trips we've done in a given month",
      "parameterized": {
        "parameterized_intent": "How many trips we've done in $1",
        "parameterized_sql": "SELECT COUNT(*) FROM truck_trips WHERE departure_time >=TO_DATE($1, 'Month YYYY') AND departure_time < TO_DATE($1, 'Month YYYY') + INTERVAL '1 month'"
      }
    }
  ]
}
```

然後使用下載按鈕，從 Cloud Shell 將範本下載到電腦。

載入 QueryData 脈絡資料集

如要使用 QueryData 脈絡資料集，請將其上傳至資料庫。

開啟 AlloyDB Studio。左側面板底部會顯示 `QueryData Context` 和三個點。



按一下這三個點，然後選擇「建立內容」。系統會開啟對話方塊，讓你輸入

- 名稱： `cymbal_context_set`
- 說明： `Cymbal Logistic Query Data`
- 上傳結構定義檔案：按一下「Browse」按鈕，然後選擇含有 QueryData ContextSet 的 JSON 檔案

如果您是第一次按下儲存按鈕，系統可能需要一些時間初始化內容儲存空間。

您應該會看到已下載的內容，點選右側的三個垂直按鈕，即可查看可用的動作。我們將在下一章從「測試環境」動作開始。

步驟 10 · 約 10 分鐘

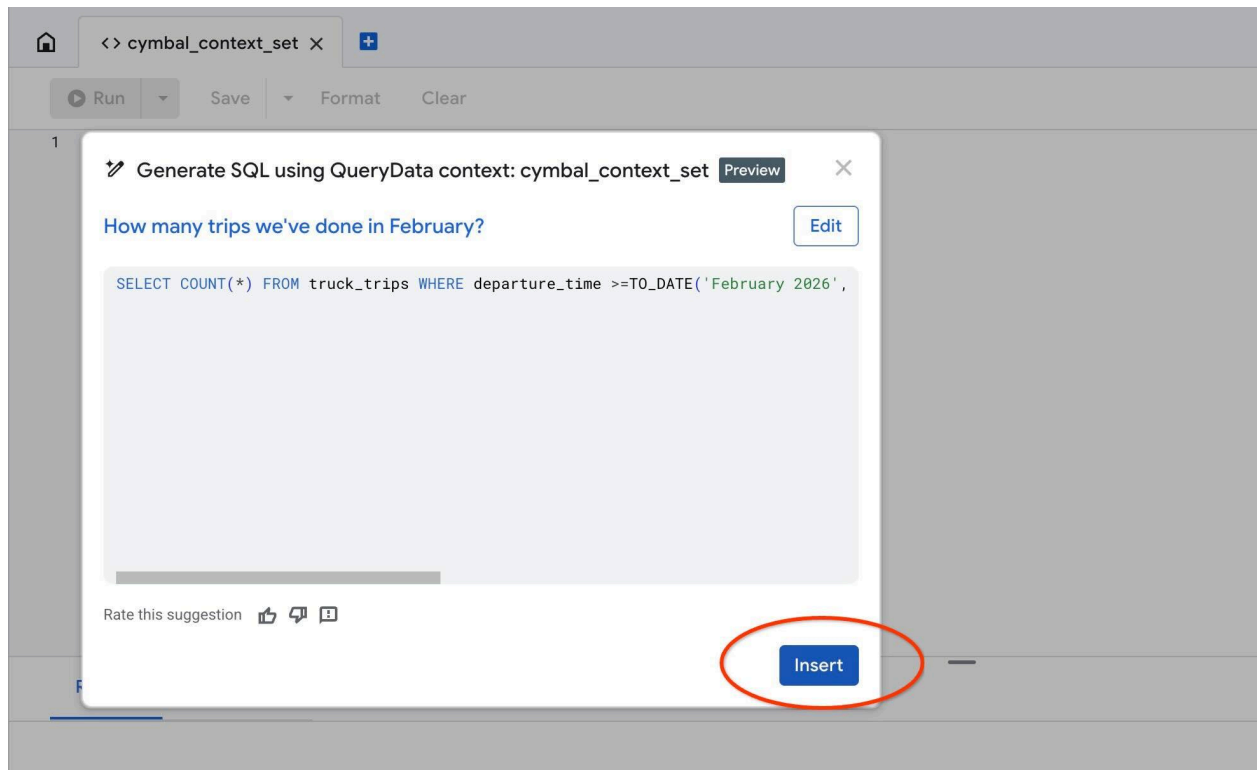
10. 測試 QueryData 脈絡資料集

測試範本

使用「Test context」動作，在 AlloyDB Studio 中測試我們的內容。按一下「測試環境」會開啟新的 AlloyDB Studio 編輯器視窗，標題為「cymbal_context_set」，並顯示「Generate SQL using QueryData context: cymbal_context_set」 Gemini SQL 生成邀請。按一下 SQL 生成，然後輸入

Hello, can you tell me how many trips we've done in February?

產生 SQL 後，請按下「Insert」按鈕。



您會看到與先前在內容範本中輸入的查詢完全相同的內容。

```
-- How many trips we've done in February?
SELECT COUNT(*) FROM truck_trips WHERE departure_time >=TO_DATE('February 2026',
'Month YYYY') AND departure_time < TO_DATE('February 2026', 'Month YYYY') + INTERVAL
'1 month'
```

請嘗試將月份替換為「January」，並檢查產生的 SQL 陳述式。系統會將月份做為參數化意圖的參數，並自動調整 SQL 陳述式。

建構 QueryData 構面

我們嘗試使用查詢範本，如果知道預期使用者要求類型，範本就能正常運作。但有時，我們偏好使用特定順序或子句來重新定義意圖，因此只引導查詢的一部分 (例如條件或篩選器) 會很有幫助。

舉例來說，如果我們要求傳回「上個月」的資料，我們希望取得上個曆月的報表 (從該月 1 號到最後一天)，而不是過去 30 天的報表。

我們可以將這類 facet 新增為 SQL 片段，連同先前新增的範本一併加入 ContextSet 設定。開啟 querydata_cymbal_contextset.json。

```
edit ~/$REPO_NAME/$SOURCE_DIR/data_agent/querydata_cymbal_contextset.json
```

並在現有範本後方加入切面。檔案中的結果內容應如下所示

```
{
  "templates": [
    {
      "nl_query": "How many trips we've done in February?",
      "sql": "SELECT COUNT(*) FROM truck_trips WHERE departure_time >=TO_DATE('February 2026', 'Month YYYY') AND departure_time < TO_DATE('February 2026', 'Month YYYY') + INTERVAL '1 month'",
      "intent": "Count trips done in a certain month like February 2026",
      "manifest": "How many trips we've done in a given month",
      "parameterized": {
        "parameterized_intent": "How many trips we've done in $1",
        "parameterized_sql": "SELECT COUNT(*) FROM truck_trips WHERE departure_time >=TO_DATE($1, 'Month YYYY') AND departure_time < TO_DATE($1, 'Month YYYY') + INTERVAL '1 month'"
      }
    },
    {
      "sql_snippet": "departure_time >=date_trunc('month', current_date - interval '1 month') AND departure_time < date_trunc('month', current_date)",
      "intent": "last month",
      "manifest": "Records for the previous calendar month",
      "parameterized": {
        "parameterized_intent": "previous calendar month",
        "parameterized_sql_snippet": "departure_time >=date_trunc('month', current_date - interval '1 month') AND departure_time < date_trunc('month', current_date)"
      }
    }
  ],
  "facets": [
    {
      "sql_snippet": "departure_time >=date_trunc('month', current_date - interval '1 month') AND departure_time < date_trunc('month', current_date)",
      "intent": "last month",
      "manifest": "Records for the previous calendar month",
      "parameterized": {
        "parameterized_intent": "previous calendar month",
        "parameterized_sql_snippet": "departure_time >=date_trunc('month', current_date - interval '1 month') AND departure_time < date_trunc('month', current_date)"
      }
    }
  ]
}
```

儲存檔案並上傳到電腦。

然後使用「查詢內容」內容動作「編輯內容」，上傳修改過後的檔案，以新內容集取代舊內容集。

現在請再次使用測試環境，並使用「上個月」意圖生成 SQL 陳述式。舉例來說，如果您為「show trucks trips for the last month」產生 SQL，系統會使用我們在 cymbal_context.json 檔案中提供的條件做為層面。

您應該會看到類似下列的內容：

```
-- show trucks trips for the last month
SELECT COUNT(*) FROM truck_trips WHERE departure_time >=date_trunc('month',
current_date - interval '1 month') AND departure_time < date_trunc('month',
current_date)
```

現在，如何搭配 AI 代理使用呢？在下一章中，我們會為 AI 代理提供「查詢資料」脈絡。

11. 使用 AI 代理查詢資料

您將使用相同的資料代理，但現在 MCP 工具箱會設定為使用 QueryData ContextSet。

準備及啟動 MCP Toolbox for Databases

我們需要 MCP Toolbox 的新設定檔，該工具會使用 Gemini Data Analytics API 和 AlloyDB 做為資料庫來源。

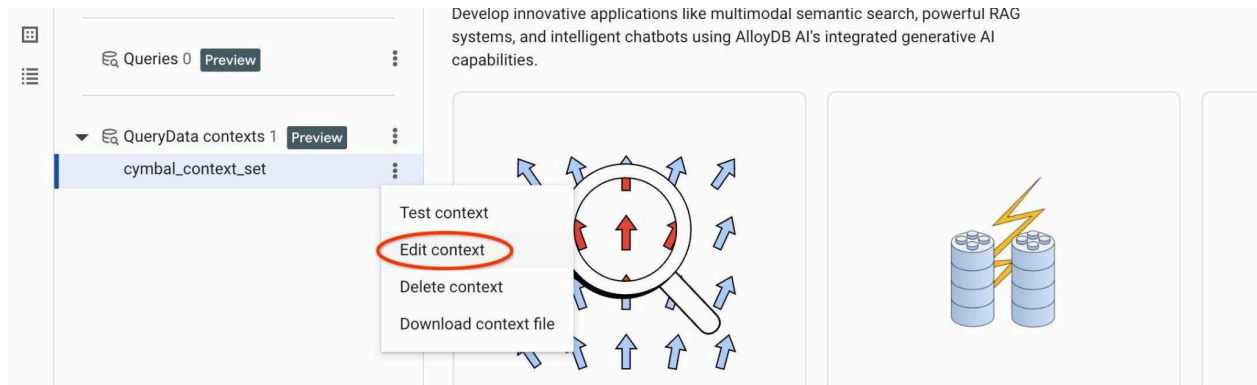
在終端機中執行：

```
PROJECT_ID=$(gcloud config get-value project)
REGION=asia-southeast3
sed -e "s/##PROJECT_ID##/$PROJECT_ID/g" \
    -e "s/##REGION##/$REGION/g" \
    querydata.yaml.example > querydata.yaml
```

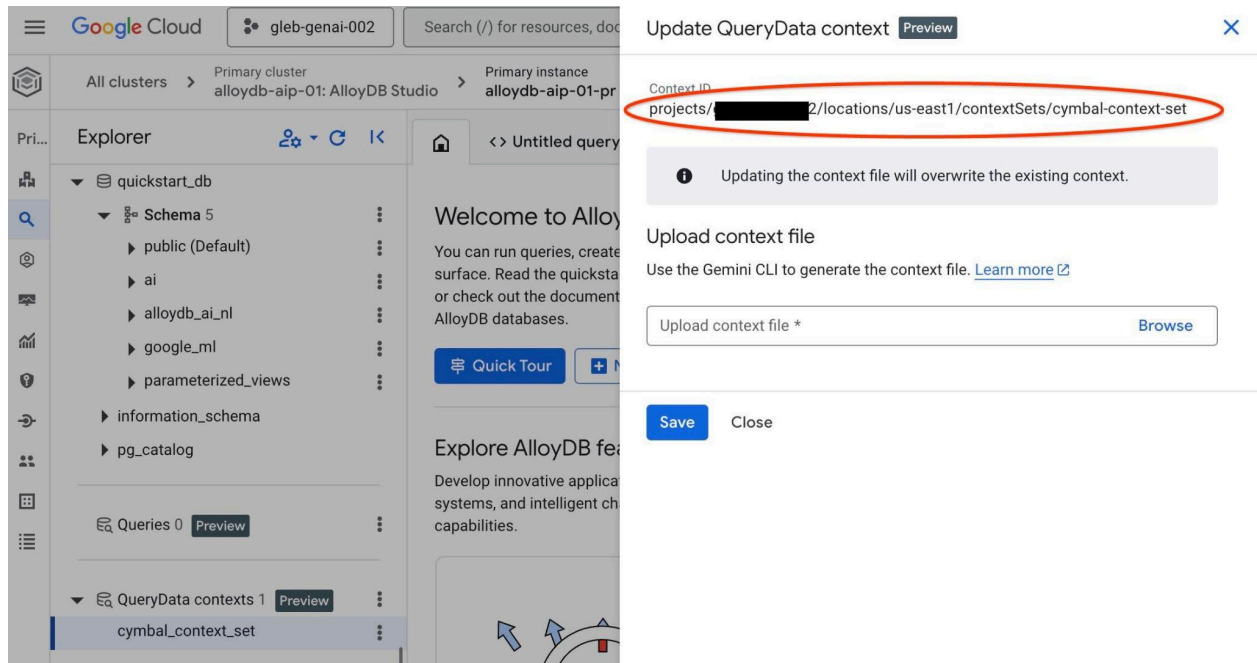
切換至編輯器，找出 `querydata.yaml` 檔案。設定檔 `querydata.yaml` 應如下所示，但專案 ID 和區域會反映您的環境。但您仍須更新 `contextSetId` 值，並將 `"<add-context-set-id>"` 預留位置替換為控制台中的值。

```
kind: sources
name: gda-api-source
type: cloud-gemini-data-analytics
projectId: test-project-001-402417
---
kind: tools
name: cloud_gda_query_tool
type: cloud-gemini-data-analytics-query
source: gda-api-source
location: "asia-southeast3"
description: Use this tool to send natural language queries to the Gemini Data Analytics API
and receive SQL, natural language answers, and explanations.
context:
  datasourceReferences:
    alloydb:
      databaseReference:
        projectId: "test-project-001-402417"
        region: "asia-southeast3"
        clusterId: "alloydb-aip-01"
        instanceId: "alloydb-aip-01-pr"
        databaseId: "quickstart_db"
      agentContextReference:
        contextSetId: "<add-context-set-id>"
  generationOptions:
    generateQueryResult: true
    generateNaturalLanguageAnswer: true
    generateExplanation: true
    generateDisambiguationQuestion: true
```

如要找出 ContextSet ID，請按一下內容集編輯按鈕，如圖片所示。



右側新分頁的頂端會顯示脈絡集 ID。



請將該完整路徑替換為 `querydata.yaml` 檔案中的 `"<add-context-set-id>"` 預留位置。

切換回終端機。

在 Google Cloud Shell 介面頂端按下「+」按鈕，開啟 Google Cloud Shell 的新分頁。

在新分頁中，切換至含有工具箱二進位檔案和設定檔 `tools.yaml` 的目錄，然後啟動 MCP 伺服器。

```
REPO_NAME="codelabs"
SOURCE_DIR="alloydb-querydata"
cd ~/$REPO_NAME/$SOURCE_DIR
./toolbox --config querydata.yaml
```

執行 ADK 代理

在第一個 Cloud Shell 分頁中啟動代理程式。

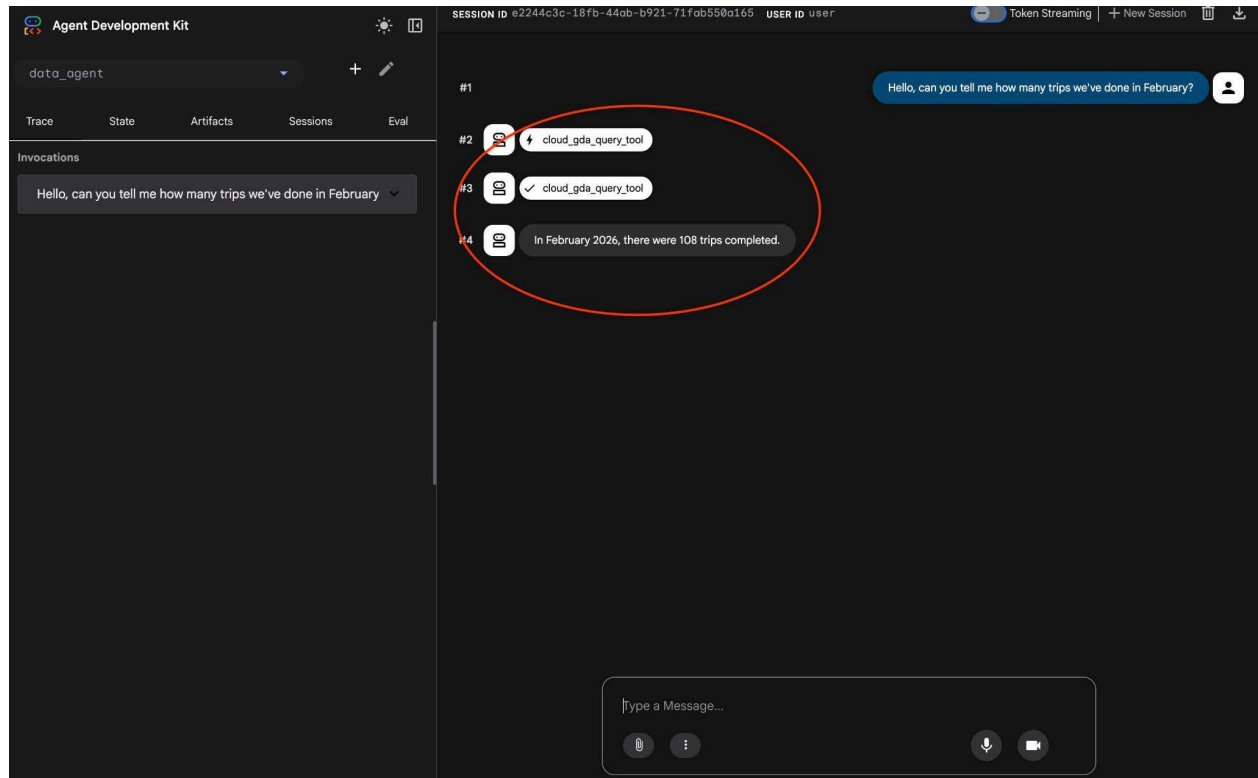
```
uv run adk web --allow_origins 'regex:https://.*\.cloudshell\.dev'
```

啟動後，再次點選 <http://127.0.0.1:8000> 連結。

您會看到熟悉的 ADK 網頁預覽代理介面。張貼與上次完全相同的查詢。

Hello, can you tell me how many trips we've done in February?

並查看代理工作流程。如果一切設定正確，您應該會看到如下所示的內容。



上次需要多個回合的要求已轉換為對 MCP 工具的一次呼叫，並使用可預測的 SQL 陳述式執行。

您可以透過下列要求測試設定的層面：

how trucks trips for the last month

在輸出內容中，如果點選工具動作，您會看到系統使用相同工具，並套用層面來取得結果。

Event 7 of 8

Event

root_agent

cloud_gda_query_tool

content:

parts:

0:

functionResponse:

id: "adk-6d7d604d-983d-438f-97af-5e00499d70df"

name: "cloud_gda_query_tool"

response:

result: "{generated_query\":\"SELECT COUNT(*) FROM truck_trips WHERE departure_time \u003e= date_trunc('month', current_date - interval '1 month') AND departure_time \u003c date_trunc('month', current_date)\", \"intent_explanation\":\"I identified Template 1 as the closest match for counting or listing trips in a specific month. I extracted 'February 2026' from the question to populate the template's parameters. Since the user specifically asked for 'last month', I applied Facet 1 to ensure the time range logic matches the 'last month' requirement relative to the current date. The resulting SQL selects the count of trips for February 2026 using the date range logic from the facet. The intent and manifest were updated to reflect both the specific month and the 'last month' context.\"\", \"query_result\": {\"columns\": [{\"name\": \"count\", \"type\": \"INT8\"}], \"rows\": [{\"values\": [{\"value\": \"108\"}]}], \"total_row_count\": 1}, \"natural language answer\": \"There were 108 truck trips in the last month according to the query. However, the provided query counts trips for the last calendar month relative to the current date, not specifically for February 2026. To confirm trips for February 2026, the 'departure_time' filter would need to be adjusted to specifically target that month.\"\"}

role: "user"

invocationId: "e-d2c0161a-9d54-4be3-98d2-69a87cb8c863"

author: "root_agent"

actions:

stateDelta:

artifactDelta:

requestedAuthConfigs:

requestedToolConfirmations:

SESSION ID e2244c3c-18fb-44ab-b921-71fab550a165

USER ID user

Token Streaming

+ New Session

#1

Hello, can you tell me how many trips we've done in February?

#2

cloud_gda_query_tool

#3

cloud_gda_query_tool

#4

In February 2026, there were 108 trips completed.

#5

show trucks trips for the last month

#6

cloud_gda_query_tool

#7

cloud_gda_query_tool

#8

There were 108 truck trips recorded for the last month.

Type a Message...

實驗室到此結束。希望您已完成所有範例，並瞭解如何使用 AlloyDB 的 QueryData。這項技術可協助您預測及可靠地生成代理工作負載和 SQL。

12. 清理環境

為避免產生非預期的費用，建議您清理暫時資源。最可靠的方法是刪除您用來測試工作流程的專案。但您也可以選擇刪除個別資源 (例如 AlloyDB)，藉此限制用量。

完成實驗室後，請終止 AlloyDB 執行個體和叢集。

刪除 AlloyDB 叢集和所有執行個體

如果您使用過 AlloyDB 試用版，如果您打算使用試用叢集測試其他實驗室和資源，請勿刪除試用叢集。您無法在同一個專案中建立其他試用叢集。

使用 force 選項終止叢集，這也會刪除叢集中的所有執行個體。

如果連線中斷，且所有先前的設定都遺失，請在 Cloud Shell 中定義專案和環境變數：

```
gcloud config set project <your project id>
```

```
export REGION=asia-southeast3
export ADBCLUSTER=alloydb-aip-01
export PROJECT_ID=$(gcloud config get-value project)
```

刪除叢集：

```
gcloud alloydb clusters delete $ADBCLUSTER --region=$REGION --force
```

注意：這個指令需要 3 到 5 分鐘才能執行完畢

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ gcloud alloydb clusters delete
$ADBCLUSTER --region=$REGION --force
All of the cluster data will be lost when the cluster is deleted.

Do you want to continue (Y/n)?  Y

Operation ID: operation-1697820178429-6082890a0b570-4a72f7e4-4c5df36f
Deleting cluster...done.
```

刪除 AlloyDB 備份

刪除叢集的所有 AlloyDB 備份：

注意：這個指令會毀損環境變數中指定名稱的叢集的所有資料備份

```
for i in $(gcloud alloydb backups list --filter="CLUSTER_NAME:
projects/$PROJECT_ID/locations/$REGION/clusters/$ADBCLUSTER" --format="value(name)" --sort-
by=~createTime) ; do gcloud alloydb backups delete $(basename $i) --region $REGION --quiet;
done
```

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ for i in $(gcloud alloydb backups
list --filter="CLUSTER_NAME:
projects/$PROJECT_ID/locations/$REGION/clusters/$ADBCLUSTER" --format="value(name)" -
-sort-by=~createTime) ; do gcloud alloydb backups delete $(basename $i) --region
$REGION --quiet; done
Operation ID: operation-1697826266108-60829fb7b5258-7f99dc0b-99f3c35f
Deleting backup...done.
```

步驟 13 · 約 1 分鐘

13. 恭喜

恭喜您完成本程式碼研究室。

涵蓋內容

- 如何建立 AlloyDB 叢集並匯入範例資料
 - 如何啟用 AlloyDB Data Access API
 - 如何為 AlloyDB 啟用 QueryData
 - 如何生成範本
 - 如何使用多面向搜尋
 - 如何搭配 AI 代理使用 QueryData
-

步驟 14 · 約 1 分鐘

14. 問卷調查

輸出內容：
